

Fraudly Agentic LMS

Répartition détaillée des tâches — Phase 1 · Semaines 1 & 2

✓ Cadrage & analyse des besoins — déjà terminé · Cahier des charges validé

P1

Développeur 1 — Architecte technique

Définit l'architecture globale · ses livrables débloquent toute l'équipe

Architecture

Semaine 1

Jours 1 – 3

Conception de l'architecture microservices

Définir les 5 services, leurs responsabilités, et comment ils communiquent entre eux.

- Nommer et délimiter les 5 services : Learning, Assessment, Proctoring, AI Agent, Analytics. Pour chacun : ce qu'il fait et ce qu'il ne fait PAS.
- Choisir les modes de communication : REST synchrone pour les appels directs, Kafka/RabbitMQ asynchrone pour les événements.
- Rédiger l'ADR : documenter chaque choix technique majeur (pourquoi Spring Boot, PostgreSQL, AWS) avec les alternatives rejetées.
- Diagramme de composants : vue d'ensemble des 5 services avec flèches de dépendances et API Gateway en entrée.

Livable ADR v1 + diagramme de composants

Jours 3 – 5

Rédiger les contrats API REST (OpenAPI 3.0)

5 fichiers openapi.yaml décrivant exactement ce que chaque service expose. Contrat entre P3 (backend) et P5 (frontend).

- Un fichier openapi.yaml par service : endpoints, verbes HTTP, schémas JSON requête/réponse, codes d'erreur (400, 401, 403, 404, 500).
- Fixer les conventions communes : format des dates (ISO 8601), pagination (page/size), format des erreurs, nommage camelCase.
- Priorité S1 : /auth/login, /auth/refresh, /users, /courses, /courses/{id}/chapters. Endpoints examen et proctoring en S2.

Livable 5 fichiers openapi.yaml (auth + cours complets)

■ Bloquant pour P5 qui en a besoin dès S2 pour configurer ses appels HTTP.

Semaine 2

Jours 6 – 8

Schéma de base de données global (ERD + DDL SQL)

Modéliser toutes les tables du projet en un schéma cohérent. P3 en a besoin pour ses migrations Flyway.

- Domaine Utilisateurs : users, roles, user_roles. Champs RGPD : deleted_at, consent_given_at.
- Domaine Cours : courses, chapters, resources. Prévoir versioning des contenus.
- Domaine Évaluations : exams, questions, answers, student_exams (score + timestamps).
- Domaine Proctoring : proctoring_sessions, fraud_events (metadata JSONB pour flexibilité).
- Exporter en DDL SQL : scripts CREATE TABLE avec contraintes FK, NOT NULL, UNIQUE et index.

Livrable ERD complet (dbdiagram.io) + scripts DDL SQL

■ Bloquant pour P3 qui attend ce livrable pour ses migrations Flyway.

Jours 8 – 10

Diagrammes UML de séquence et de déploiement

Documenter les flux critiques et l'architecture cloud AWS pour la documentation technique finale.

- Séquence Auth : Frontend → Gateway → Auth Service → BDD, avec refresh token et session expirée.
- Séquence Examen : démarrage → proctoring actif → sauvegarde auto 30s → soumission → correction.
- Séquence Génération QCM : upload PDF → Knowledge Service → LLM → banque de questions.
- Déploiement AWS : EC2/EKS, RDS, S3, load balancer, security groups, pods Kubernetes.

Livrable 4 diagrammes UML finalisés (PlantUML / Lucidchart)

P2

Développeur 2 — DevOps / Cloud AWS

Infrastructure, déploiement, CI/CD et monitoring

DevOps

Semaine 1

Jours 1 – 3

Provisionnement infrastructure AWS

Créer et configurer l'environnement cloud complet avec Terraform pour avoir une infra reproductible et versionnée.

- IAM : créer les rôles et politiques least-privilege pour chaque service (EC2, RDS, S3, EKS).
- Réseau VPC : subnets publics/privés, security groups, ACLs. Microservices en subnets privés, Gateway seule en public.
- RDS PostgreSQL : instance dev (single-AZ), instance staging (multi-AZ). Backups automatiques configurés.
- S3 Buckets : ressources pédagogiques (PDFs, vidéos), logs d'audit, artefacts CI/CD.

Livrable Infrastructure AWS opérationnelle (scripts Terraform)

Jours 3 – 5

Dockerisation de tous les services

Containeriser chaque service pour garantir un environnement identique entre dev, staging et prod.

- Dockerfile Spring Boot : multi-stage build (Maven → JRE slim). Image finale < 200MB.
- Dockerfile FastAPI : Python 3.11-slim, dépendances IA (torch, transformers, langchain).
- Dockerfile Frontend : build Angular/Next.js → image nginx pour fichiers statiques.
- docker-compose.yml : lancer tout le projet en local avec une seule commande. README inclus.

Livable Dockerfiles + docker-compose.yml + README

Semaine 2

Jours 6 – 8

Pipeline CI/CD complet

Automatiser le cycle build → test → déploiement pour que chaque push déclenche une mise à jour sur staging.

- GitHub Actions : jobs lint, tests unitaires, build Docker, push ECR, déploiement EKS sur staging.
- Gates de qualité : couverture code minimum 70%, aucun secret en dur, scan vulnérabilités Docker (Trivy).
- Stratégie branches : main (prod), develop (staging), feature/* (dev). Merge uniquement via Pull Request.
- Notifications : alertes Slack sur échec du pipeline, rapport couverture automatique sur chaque PR.

Livable Pipeline CI/CD actif sur 3 branches (main, develop, feature)

Jours 8 – 10

Monitoring, logging et alertes

Mettre en place l'observabilité complète pour détecter les problèmes avant qu'ils impactent les utilisateurs.

- CloudWatch : métriques AWS (CPU, mémoire, latence RDS, taille S3). Dashboards par service.
- Logs centralisés : ELK Stack (Elasticsearch + Logstash + Kibana). Tous les services envoient des logs JSON.
- Alertes : latence HTTP > 500ms, taux d'erreur > 1%, espace disque RDS > 80%, pod Kubernetes en crash loop.

Livable Dashboard monitoring + alertes configurées

P3

Développeur 3 — Backend Java / Spring Boot

Microservices métier Java, couche données et authentification

Java / Spring

Semaine 1

Jours 1 – 3

Scaffolding des 3 microservices Spring Boot

Initialiser les projets Java avec toutes les dépendances et la structure standard avant d'écrire la moindre ligne métier.

- 3 projets via Spring Initializr : Learning, Assessment, Proctoring. Dépendances : Spring Web, Security, Data JPA, Flyway, Actuator, Lombok.
- Module commun partagé : DTOs communs, exceptions personnalisées (ResourceNotFoundException), utilitaires pagination.
- Configuration multi-env : profils Spring dev/staging/prod avec application-{env}.yml. Secrets via AWS Secrets Manager.
- Documentation Swagger : springdoc-openapi pour UI automatique sur /swagger-ui.html dans chaque service.

Livable 3 projets Spring Boot compilables + README d'onboarding

Jours 3 – 5

Authentification JWT — fondations complètes

Implémenter le système d'auth sécurisé qui sera utilisé par toutes les phases suivantes.

- Filtre JWT : JwtAuthenticationFilter intercepte chaque requête, valide le token et charge le contexte Spring Security.
- Endpoints auth : POST /auth/login (access token 15min + refresh token 7j), POST /auth/refresh, POST /auth/logout.
- Gestion des rôles : ROLE_STUDENT, ROLE_TEACHER, ROLE_ADMIN. Annotations @PreAuthorize sur endpoints sensibles.
- Tests unitaires : cas valide, token expiré, token falsifié, rôle insuffisant. Couverture > 80% sur le module auth.

Livable Auth JWT fonctionnelle + tests unitaires > 80%

Semaine 2

Jours 6 – 8

Migrations Flyway + entités JPA

Implémenter le schéma BDD en migrations versionnées et les entités Java correspondantes. Se base sur l'ERD de P1.

- Migration V1 : tables users, roles, user_roles avec contraintes et index.
- Migration V2 : tables courses, chapters, resources. Clé étrangère vers users (teacher_id).
- Migration V3 : tables exams, questions, answers, student_exams.
- Entités JPA : classes annotées @Entity avec @OneToMany, @ManyToMany, validations Bean Validation et repositories Spring Data.

Livable Migrations V1-V3 + entités JPA + repositories

■ Dépend de l'ERD de P1 (livable semaine 2). Coordonner dès J6.

Jours 8 – 10

Endpoints CRUD de base — Learning Service

Implémenter les premiers endpoints métier pour que P5 puisse commencer à les brancher côté frontend.

- GET/POST/PUT/DELETE /courses : liste des cours de l'enseignant connecté, création, modification, suppression.
- GET/POST /courses/{id}/chapters : gestion des chapitres d'un cours avec ordre.
- GET /users/me : profil de l'utilisateur connecté (nom, email, rôle). Utilisé par P5 pour l'affichage du dashboard.

Livrable Endpoints cours + profil testables via Swagger

P4

Développeur 4 — Backend Python / IA

Service IA multi-agents, LLM, Vector DB et pipeline RAG

Python / IA

Semaine 1

Jours 1 – 3

Scaffolding FastAPI + structure des agents

Initialiser le service IA avec une architecture modulaire qui accueillera les 6 agents des phases suivantes.

- Structure FastAPI : dossiers routers/, services/, agents/, models/, core/. Chaque agent aura son propre module isolé.
- Dépendances IA : LangChain, LangGraph (orchestration agents), Transformers (HuggingFace), PyTorch. requirements.txt versionné.
- Logging structuré : logs JSON avec niveau, timestamp, agent_name, action. Compatibles avec ELK de P2.
- Healthcheck + Swagger : endpoint /health, documentation automatique FastAPI sur /docs.

Livrable Projet FastAPI structuré, déployable sur Docker de P2

Jours 3 – 5

Spike LLM — benchmark et choix définitif

Tester les modèles LLM pour choisir le bon avant de construire quoi que ce soit dessus. Évite de repartir de zéro en phase 3.

- Tester LLaMA 3 via Ollama (local) ou API Together.ai : latence, qualité des QCM produits, coût par token.
- Tester Mistral 7B : mêmes critères. Mistral souvent plus rapide, LLaMA 3 souvent plus précis en français.
- Critères de décision : latence < 2s (temps réel Tutor Agent), qualité questions générées, coût mensuel pour 500 étudiants.
- Rapport de spike : tableau comparatif avec le choix retenu et la justification. Partagé avec toute l'équipe.

Livrable Rapport spike LLM + choix validé par l'équipe

Semaine 2

Jours 6 – 8

Pipeline RAG bout-en-bout (Knowledge Service)

Implémenter le pipeline complet : PDF uploadé → texte extrait → découpé → vecteurs → stocké dans Vector DB → requêtable.

- Configurer Pinecone ou Weaviate : créer l'index, définir la dimension des vecteurs selon le modèle d'embedding retenu.
- Découpage en chunks : LangChain RecursiveCharacterTextSplitter, taille 512 tokens, overlap 50 tokens.
- Génération d'embeddings : transformer chaque chunk en vecteur via sentence-transformers/paraphrase-multilingual.
- Test end-to-end : uploader un PDF réel, poser une question, vérifier que la réponse LLM est basée sur le PDF.

Livrable Pipeline RAG fonctionnel testé sur un PDF réel

Jours 8 – 10

Squelette des 6 agents IA + orchestrateur v0

Créer la structure de base de chaque agent sans encore implémenter la logique métier. Prépare proprement la phase 3.

- Classes abstraites LangGraph : BaseAgent avec méthodes run(), handle_message(), get_status(). 6 agents héritent de BaseAgent.
- Bus de communication : système de messages inter-agents (un agent peut demander des infos à un autre). Interfaces uniquement.
- Orchestrateur v0 : LangGraph StateGraph définissant les transitions entre agents. Ex: requête étudiant → Tutor → Knowledge → réponse.

Livrable 6 classes agents + orchestrateur LangGraph v0

P5

Développeur 5 — Frontend Angular / Next.js

Interface utilisateur, design system et intégration HTTP

Frontend

Semaine 1

Jours 1 – 3

Scaffolding frontend + design system

Initialiser le projet et créer la bibliothèque de composants réutilisables que toutes les pages utiliseront.

- Init Angular / Next.js : structure features/, shared/, core/. Configurer ESLint, Prettier, Husky (pre-commit hooks).
- Tailwind CSS : définir la palette de couleurs Fraudly, la typographie et les espacements dans tailwind.config.js.
- Composants de base : Button (variants primary/secondary/danger), Input, Modal, Toast, Badge, Card, Loader, Avatar. Documentés dans Storybook.
- Routing principal : AuthModule, DashboardModule, CourseModule, ExamModule, AdminModule. Guards de routes en squelette.

Livrable Projet frontend + 10+ composants UI dans Storybook

Jours 3 – 5

Maquettes Figma des 6 écrans principaux

Dessiner les wireframes haute-fidélité avant de coder les pages, pour valider les flux UX avec l'équipe.

- Écran 1 — Login/Register : formulaires validés, lien mot de passe oublié, redirection selon rôle après connexion.
- Écran 2 — Dashboard enseignant : liste cours, statistiques rapides (nb étudiants, taux réussite), accès création examen.
- Écran 3 — Gestion d'un cours : liste chapitres, upload ressources, drag & drop pour réordonner.
- Écran 4 — Interface d'examen étudiant : mode plein écran, chronomètre, questions, sauvegarde automatique visible.
- Écran 5 — Dashboard analytics : graphiques performances, distribution notes, alertes fraude.
- Écran 6 — Interface admin : gestion utilisateurs, supervision examens en cours, accès aux logs.

Livable 6 maquettes Figma haute-fidélité validées par l'équipe

Semaine 2

Jours 6 – 8

State management + service HTTP

Mettre en place la gestion d'état globale et la couche de communication avec les APIs backend.

- NgRx / Zustand : store global pour l'authentification (user, token, rôle). Actions : login, logout, refreshToken, setUser.
- Service HTTP : classe ApiService centralisée avec méthodes get(), post(), put(), delete(). Proxy dev vers localhost:8080.
- Intercepteur JWT : injecte Authorization: Bearer {token} sur chaque requête. Refresh automatique si token expiré (401 → /auth/refresh → retry).
- Guards de routes : AuthGuard (redirige vers /login si non connecté), RoleGuard (redirige si rôle insuffisant).

Livable State management + HTTP + guards opérationnels

■ Dépend des fichiers openapi.yaml de P1 pour connaître les endpoints exacts.

Jours 8 – 10

Intégration Auth E2E + pages login et dashboard

Brancher le frontend sur les endpoints auth de P3 et implémenter les deux premières pages réelles de l'application.

- Page Login : formulaire connecté à POST /auth/login, gestion erreurs (mauvais identifiants), redirection selon rôle.
- Page Dashboard enseignant : appel GET /courses pour lister les cours, affichage avec les composants Card créés en S1.
- Test flux complet : login → token stocké → appel API protégé → affichage données → logout → redirection login.

Livable Flux auth E2E fonctionnel + pages login et dashboard

■ Dépend des endpoints /auth/login et /courses de P3 (disponibles fin S2).

Récapitulatif des livrables

Personne	Rôle	Tâches	Livrables clés
P1 — Architecte	Architecture globale	4	ADR · 5 openapi.yaml · ERD + DDL · 4 UML
P2 — DevOps	Infrastructure & CI/CD	4	Terraform AWS · Dockerfiles · Pipeline CI/CD · Monitoring
P3 — Backend Java	Spring Boot & BDD	4	3 services Spring · Auth JWT · Migrations V1–V3 · CRUD cours
P4 — Backend IA	FastAPI & Agents IA	4	FastAPI structuré · Rapport spike LLM · Pipeline RAG · Agents v0
P5 — Frontend	Angular / Next.js	4	Design system · 6 maquettes Figma · HTTP + guards · Auth E2E

P3 présente : auth JWT E2E + migrations BDD + endpoints /courses testables.

P4 présent